

DetectCore - Engineer-Friendly Overview

Overview

DetectCore is a lightweight C++ library for evaluating whether an object is detectable by radar. It uses a configurable pipeline that computes radar features (such as SNR and JNR) and runs detection algorithms to produce a final result.

High-Level Workflow

Context -> Model.run(context) -> Pipeline -> Compute Features (on demand) -> Cache Features -> Run Evaluators -> Result

Context

Represents the input data for a single object at a specific point in time (radar parameters, target state, environment, etc.).

FeatureExtractor

Computes one FeatureValue from the Context (for example SNR or JNR). FeatureExtractors may depend on other features and declare those dependencies using requires().

FeatureRegistry

Maps each FeatureId to the FeatureExtractor capable of producing it. The pipeline uses it to locate the correct extractor.

FeatureState

Caches computed FeatureValues during a single run. A feature is computed once and reused by all evaluators.

Evaluator

Consumes FeatureValues and produces an Evaluation (for example whether the target is detectable). Evaluators do not compute features themselves.

Config

Enables or disables FeatureExtractors and Evaluators and defines execution Stage and Order.

Pipeline

Coordinates feature computation, dependency resolution, caching, evaluator execution, and aggregation of the final Result.

Model

Public API. Users simply construct a Model with a Config and call model.run(context). The Model hides the pipeline implementation (Facade pattern).

Dependency Resolution

Feature dependencies are resolved automatically. If an evaluator requests a missing feature, the pipeline computes it through the appropriate FeatureExtractor and caches the result.

Stage and Order

Evaluators execute according to configured Stage and Order. The pipeline may return early when an evaluator reaches a final decision.

Summary

DetectCore is built around three ideas: (1) Lazy evaluation—features are computed only when needed; (2) Dependency resolution—feature dependencies are handled automatically; (3) Caching—each feature is computed once and reused across evaluators.